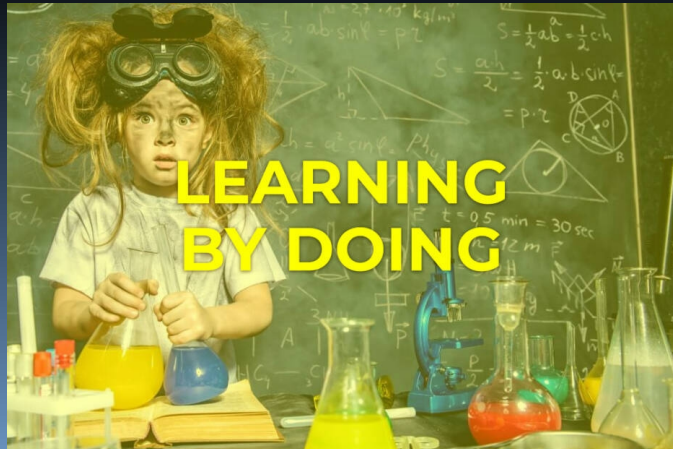




UNIVERSITÀ DI PARMA

Snake Game



For the things we have to learn before we can do them, we learn by doing them

Aristotle, The Nicomachean Ethics

*You don't learn to walk by following rules.
You learn by doing, and by falling over*

Richard Branson, Virgin Group

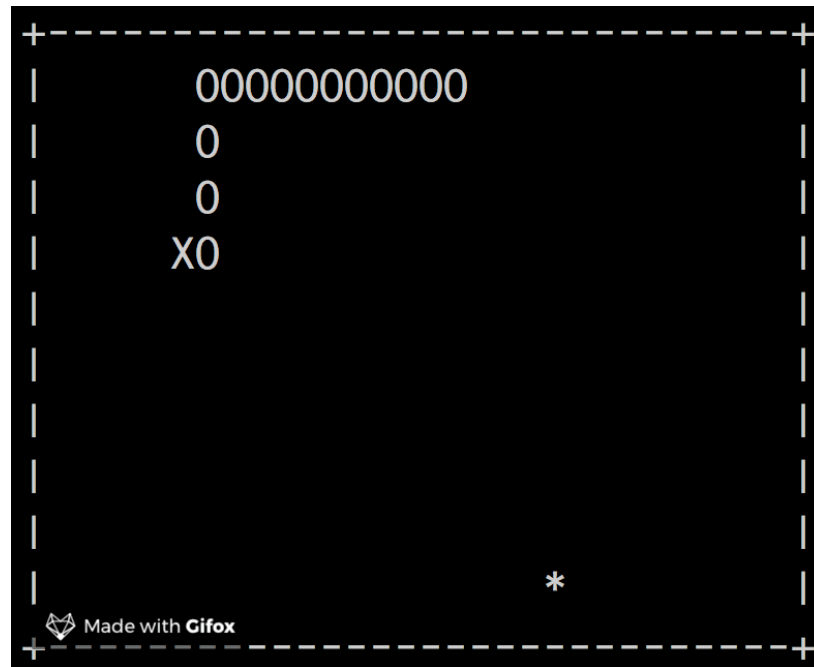
- Why this game?
- Prerequisites
- Different development steps
 - Flow control
 - Variables and constants
 - Predefined functions
 - Arrays
 - Functions
 - I/O

SUMMARY



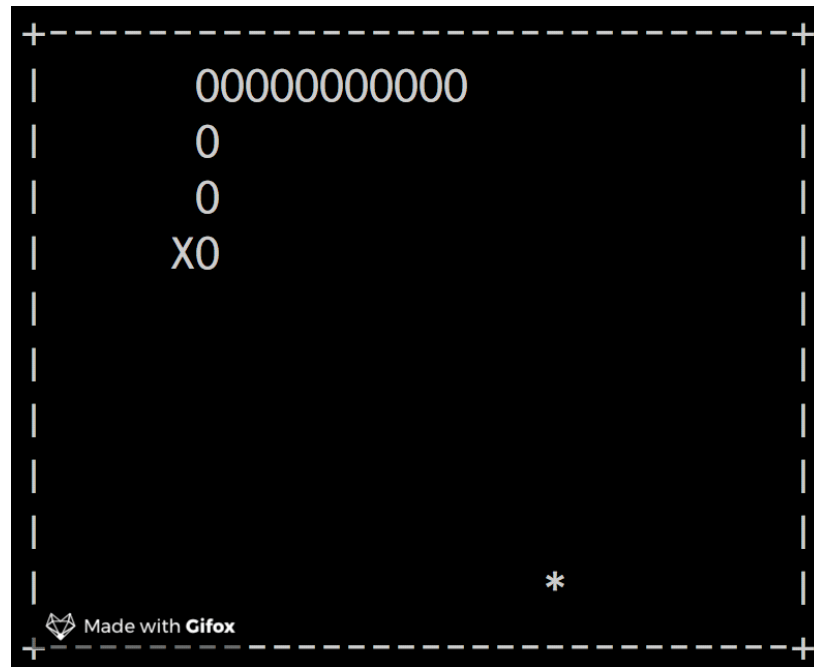
Why this game?

- The idea is giving you a project that will be continuously improved up to the end of this course
- Console based graphic
 - Back to '70s
- Please note that these slides will be continuously updated!



Why this game?

- What is the snake program?
- It is a simple game that can easily be rendered using only the console (1976)
- The player control a “snake” that is continuously moving
- Each time the “head” of the snake “eat” some “food”, the snake itself grows becoming longer and longer
- When the snake hits the “walls” or itself it is a game over!



- Even console “graphics” can be difficult
- 2 main issues:
 - In a game I would like to read keypresses without stopping the program
 - `scanf()` and other console input functions stops until a “return” is entered...
 - I also would like to be able to write in a given position
 - `printf()` and other console output functions can not behave like this...

- We will start from a C “skeleton” that contains specific functions to solve those issues
- Therefore:
 - Create a “snake” project in your folder (U:\ for Lab workstations)
 - Overwrite the main.c in your project with the skel.c in the “snake” folder in the lab repository

10. Flow control: selection

- Print a menu like:
 1. Start game
 2. Scores
 3. Help
 4. Quit
- Read from keyboard the user choice

- Print the choice (as debug) and act as follows
 - Help:
 - Print “Q: left, E: right, W: up, S: down, ‘space’: pause, X: quit”
 - Quit: end the program
 - Scores: do nothing for now
 - Start: go on to the next slide
- What is the best selection statement to use?

20. Flow Control: cycles

- Modify step #10 in order to print again the menu:
 - When the user enters a wrong choice
 - After the help
- When the user enters the “start” go on according to next slides

- Print a game field
- Namely, a 23×97 rectangle surrounded by a border
 - Use '+' for corners
 - Use '-' for horizontal borders
 - Use '|' (ASCII 124) for vertical borders
- How many cycles do we need?
- Do we need to nest some cycles?

30. Variables & Constant

- Use a variable for the score and init it to zero
- Modify 20#:
 - Print the score before the playing field using 6 characters and using zero as padding character
 - Use constants for the symbols used for the border of the playing field
 - Use constants for the size of playing field
- Use also variables for the snake head position and movement
 - Which kind of variables do we have to use? How many?



UNIVERSITÀ DI PARMA

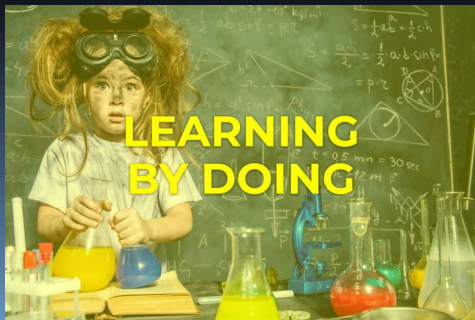
To be continued...





UNIVERSITÀ DI PARMA

Snake Game



For the things we have to learn before we can do them, we learn by doing them

Aristotle, The Nicomachean Ethics

*You don't learn to walk by following rules.
You learn by doing, and by falling over*

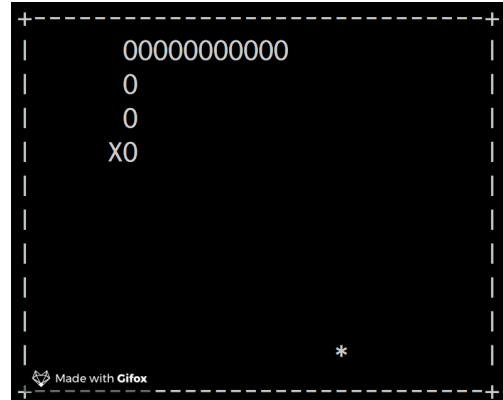
Richard Branson, Virgin Group

- Why this game?
- Prerequisites
- Different development steps
 - Flow control
 - Variables and constants
 - Predefined functions
 - Arrays
 - Functions
 - I/O

SUMMARY



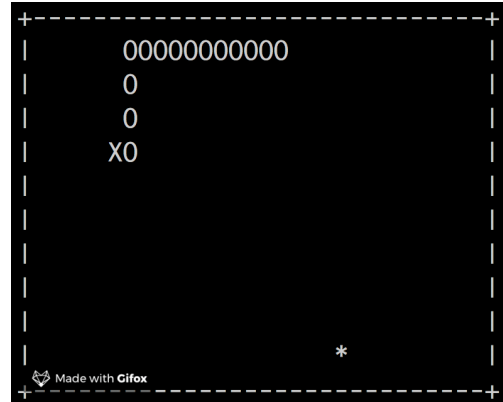
- The idea is giving you a project that will be continuously improved up to the end of this course
- Console based graphic
 - Back to '70s
- Please note that these slides will be continuously updated!



Why this game?



- What is the snake program?
- It is a simple game that can easily be rendered using only the console (1976)
- The player control a “snake” that is continuously moving
- Each time the “head” of the snake “eat” some “food”, the snake itself grows becoming longer and longer
- When the snake hits the “walls” or itself it is a game over!





- Even console “graphics” can be difficult
- 2 main issues:
 - In a game I would like to read keypresses without stopping the program
 - `scanf()` and other console input functions stops until a “return” is entered...
 - I also would like to be able to write in a given position
 - `printf()` and other console output functions can not behave like this...



- We will start from a C “skeleton” that contains specific functions to solve those issues
- Therefore:
 - Create a “snake” project in your folder (U:\ for Lab workstations)
 - Overwrite the main.c in your project with the skel.c in the “snake” folder in the lab repository

- Print a menu like:
 1. Start game
 2. Scores
 3. Help
 4. Quit
- Read from keyboard the user choice



- Print the choice (as debug) and act as follows
 - Help:
 - Print “Q: left, E: right, W: up, S: down, ‘space’: pause, X: quit”
 - Quit: end the program
 - Scores: do nothing for now
 - Start: go on to the next slide
- What is the best selection statement to use?

- Modify step #10 in order to print again the menu:
 - When the user enters a wrong choice
 - After the help
- When the user enters the “start” go on according to next slides



- Print a game field
- Namely, a 23×97 rectangle surrounded by a border
 - Use '+' for corners
 - Use '-' for horizontal borders
 - Use '|' (ASCII 124) for vertical borders
- How many cycles do we need?
- Do we need to nest some cycles?



- Use a variable for the score and init it to zero
- Modify 20#:
 - Print the score before the playing field using 6 characters and using zero as padding character
 - Use constants for the symbols used for the border of the playing field
 - Use constants for the size of playing field
- Use also variables for the snake head position and movement
 - Which kind of variables do we have to use? How many?



UNIVERSITÀ DI PARMA

To be continued...

